GATE CS — Operating System

Concise study notes for GATE 2027 · built from the 2016–2026 papers in this repo

Where the marks come from (average marks per paper, 2016–2026)

Topic	Marks / paper	Priority
Memory management, paging, virtual memory	2.5	Tier 2
File system & disk	1.5	Tier 3
Synchronization (semaphores)	1.3	Tier 3
CPU scheduling	1.3	Tier 3
Processes, threads, system calls, interrupts	1.1	Tier 3
Deadlock	0.8	Tier 3

Operating System carries about **8.5 of the 85 technical marks** in a typical paper. Topics below are listed in the order given in the notes; use the table above to decide where to spend time first.

Legend: **Trap** = common mistake · **Tip** = shortcut. For practice, use `GATE_CS_Operating_System.pdf` (past questions + new questions with solutions). These notes were written from general knowledge, not checked line by line against a textbook; verify the less common facts.

1. Memory Management, Paging, Virtual Memory

Basics

- Logical (virtual) address → physical address via the MMU. Contiguous allocation: first fit, best fit, worst fit; external fragmentation (compaction fixes it). Paging removes external fragmentation but has **internal** fragmentation (average half a page per process).
- Paging: logical address = (page number p, offset d). With page size 2^d and virtual address of v bits: p = v − d bits. Number of pages = 2^v / page size. Frame number bits = physical address bits − offset bits.
- **Page table size** = number of pages × page-table-entry size. Example: 48-bit VA, 8 KB pages, 8-byte entries → 2^{35} entries × 8 B = 256 GB for a single-level table.
- **Multilevel paging:** split the page number among levels so each page-table page fits in one frame: entries per page = page size / PTE size. For a 2-level scheme the outer table has 2^b entries; levels needed L satisfy (entries per page)^L ≥ number of pages.
- **Inverted page table:** one entry per physical frame, searched with a hash; saves space on huge address spaces.
- Segmentation: variable-size segments (logical units), address = (segment, offset) with a base/limit check; external fragmentation. Segmented paging combines both.

TLB and effective access time

```
EAT (single-level paging) = h·(t_tlb + t_mem) + (1 − h)·(t_tlb + 2·t_mem)
With page faults: EAT = (1 − p)·(memory access time) + p·(page-fault service time)
Multi-level paging: a TLB miss costs (levels + 1) memory accesses
```

- TLB: small associative cache of page-table entries; flushed (or tagged with ASID) on a context switch. The TLB reach = entries × page size.
- Page fault service steps: trap, find the page on disk, pick a victim frame, write it back if dirty (extra disk time), read the page, update tables, restart the instruction. Include a dirty-page probability in the service time when the question gives one.

Page replacement

- **FIFO**: evict the oldest; can show **Belady's anomaly** (more frames → more faults). **Optimal**: evict the page used farthest in the future (lower bound, not implementable). **LRU**: evict the least recently used; a stack algorithm (no Belady anomaly). Also: LFU, MFU, clock/second chance.
- Count faults by writing the frames after each reference; mark hits. Number of faults ≥ number of distinct pages (cold misses). Reference string with 3 frames: 2 3 2 1 5 2 4 5 3 2 5 2 → FIFO 9 faults, LRU 7, optimal 6.
- **Thrashing:** too many processes, too few frames → constant paging, CPU utilisation collapses. Remedy: working-set model, page-fault-frequency control, lower the degree of multiprogramming.
- Locality: a loop over a 2-D array in row-major order touches consecutive memory; column-major access over a big array causes many more page faults.
- Demand paging, copy-on-write (after fork), memory-mapped files, allocation policies (equal, proportional), buddy allocator (power-of-two blocks).

2. File System & Disk

Disk performance

- Access time = seek time + rotational latency + transfer time. Average rotational latency = ½ revolution = ½ · 60/RPM seconds. Transfer time = (bytes per track / data per revolution) per sector fraction: sector size / (track size) × rotation time.
- Disk scheduling: **FCFS**; **SSTF** (shortest seek first, may starve far requests); **SCAN** (elevator: go to one end, then reverse); **C-SCAN** (one-directional sweep, return without servicing); **LOOK/C-LOOK** (go only as far as the last request). Total head movement is the sum of absolute differences along the service order.
- Disk capacity = surfaces × tracks per surface × sectors per track × bytes per sector. RAID: 0 striping, 1 mirroring, 5 striping with distributed parity, 6 double parity.

File allocation

- **Contiguous:** fast sequential and direct access, external fragmentation, hard to grow. **Linked:** no external fragmentation, slow random access, pointer overhead; FAT keeps the links in a table in memory. **Indexed:** an index block lists the data blocks; supports random access; multilevel and combined (Unix inode) schemes for large files.
- **Unix inode:** a number of direct pointers plus single, double and triple indirect. Pointers per indirect block = block size / pointer size. Max file size = block size × $(D + P + P^2 + P^3)$ with the levels present (P = pointers per block, D = direct pointers).
- Free-space management: bit vector, linked list, grouping, counting. Directory structures: single-level, two-level, tree, acyclic graph (links), general graph. Hard link vs soft link (a soft link can break; a hard link raises the inode's link count).
- File operations/system calls: open, read, write, close, lseek; open-file table (per-process and system-wide).

3. CPU Scheduling

- Definitions: **Turnaround** = completion − arrival. **Waiting** = turnaround − burst. **Response** = first run − arrival. Throughput = jobs per time unit. Context-switch overhead reduces utilisation.
- **FCFS:** non-preemptive; convoy effect. **SJF:** minimum average waiting time among non-preemptive schedulers (optimal); can starve long jobs. **SRTF:** preemptive SJF, optimal for average waiting time. **Priority:** may starve low priority; fix by aging. **Round robin:** time quantum q; each process waits at most (n−1)q before getting the CPU; very large q → FCFS; very small q → overhead. **Multilevel queue / multilevel feedback queue:** processes move between queues by behaviour.
- Solve with a Gantt chart: advance time, note arrivals, pick the next job by the policy, and for preemptive policies re-check at every arrival. In RR, a new arrival is placed in the queue before the preempted process if they coincide (state the convention used).
- Realtime: rate-monotonic (static priorities by shorter period): schedulable if utilisation ≤ n(2^(1/n) − 1) (about 69% as n → ∞); EDF (earliest deadline first): schedulable iff total utilisation ≤ 1 (for independent periodic tasks with deadlines = periods).
- Schedulers: long-term (admission), short-term (CPU), medium-term (swapping). Dispatcher latency = time to stop one process and start another.

> **Tip:** For average waiting time with identical arrival times, SJF order is simply ascending burst times; waiting time of each job = sum of earlier burst times.

4. Process Synchronization

- **Critical-section problem:** mutual exclusion, progress, bounded waiting. A race condition happens when the result depends on the interleaving. Software solutions: Peterson's algorithm (two processes), Dekker. Hardware: test-and-set, compare-and-swap, atomic swap; disabling interrupts (single processor only).
- **Semaphores:** P(S) / wait: S--, block if S < 0 (or wait until S > 0); V(S) / signal: S++, wake a waiter. A binary semaphore is a mutex; a counting semaphore tracks a number of resources. Semaphore operations must be atomic.
- **Producer–consumer** (bounded buffer): semaphores empty = N, full = 0, mutex = 1. Producer: P(empty); P(mutex); put; V(mutex); V(full). Consumer: P(full); P(mutex); get; V(mutex); V(empty). Swapping P(empty) and P(mutex) in the producer can deadlock (a full buffer with the producer holding the mutex).
- **Readers–writers:** readers can share; writers need exclusive access. **Dining philosophers:** naive solution deadlocks; fixes: limit to n−1 diners, pick up both forks atomically, asymmetric order, resource ordering.
- Typical GATE question: given several processes with P/V on s1, s2, … — determine the possible output strings, whether deadlock is possible, the initial values needed to get an alternating output, or the final value of a shared variable.
- Counting semaphore values: initial value N allows N processes to pass before blocking; the number of blocked processes = −S when S < 0. With n processes each doing P then V on a semaphore initialised to k, up to k run concurrently.
- **Monitors** (condition variables wait/signal), spinlocks (busy waiting; good for short waits on multiprocessors), priority inversion (fix: priority inheritance).
- Lost-update example: two threads doing `x = x + 1` 100 times each without locking can give final x anywhere from 2 to 200 (classic worst-case analysis).

5. Processes, Threads, System Calls

- Process states: new → ready → running → waiting (blocked) → ready … → terminated. A PCB holds the PID, state, program counter, registers, scheduling info, memory info, open files. **Context switch** = save one PCB and load another; pure overhead.
- **Thread:** a unit of CPU execution inside a process. Threads share code, data, heap, open files; each has its own stack, registers and program counter. Thread switching is cheaper than process switching (no change of address space). User-level threads are managed by a library (a blocking system call blocks the whole process in many-to-one); kernel-level threads are scheduled by the OS (can run in parallel on multiple CPUs).
- **fork():** returns 0 in the child and the child's PID in the parent. n successive unconditional forks create 2^n processes ($2^n - 1$ children). For conditional forks, track which process executes which statement. Example: `if (fork()) fork(); fork(); print` prints 6 times. `wait()` makes the parent wait; an orphan is adopted by init; a zombie has terminated but is not yet reaped.
- **exec()** replaces the process image and does not return on success. **System call:** software interrupt (trap) that switches from user mode to kernel mode; the mode bit distinguishes the two. Privileged instructions (I/O, halting, changing the MMU) run only in kernel mode.
- Interrupts: hardware asynchronous events; exceptions/traps are synchronous (page fault, divide by zero, system call). The interrupt handler saves the context, runs the ISR, restores it. Interrupt-driven I/O vs polling vs DMA.
- Inter-process communication: pipes, message queues, shared memory (fast, needs synchronisation), sockets, signals.
- Kernel types: monolithic, microkernel, layered; virtual machines and hypervisors; booting.

6. Deadlock

- **Four necessary conditions** (Coffman): mutual exclusion, hold and wait, no preemption, circular wait. All four must hold for a deadlock; breaking any one prevents it.
- **Resource-allocation graph:** a cycle with single-instance resources means deadlock; with multiple instances a cycle is necessary but not sufficient.
- Prevention (break a condition; resource ordering removes circular wait), avoidance (**Banker's algorithm**: grant only if the resulting state is safe), detection and recovery (kill processes, preempt resources), or ignore (ostrich algorithm).
- **Banker's algorithm:** Need = Max − Allocation. Find a process whose Need ≤ Available; assume it finishes and add its Allocation to Available; repeat. If all processes can finish, the state is **safe** and the sequence found is a safe sequence. An unsafe state may or may not lead to deadlock; a deadlock state is always unsafe.
- Resource requests: grant a request only if Request ≤ Need and Request ≤ Available, and the state after the pretend allocation is safe.
- **Minimum resources guaranteeing no deadlock:** for n processes each needing at most m units of one resource type: $R = n(m - 1) + 1$. Example: 5 processes, max 3 each → 11. With the number of resources fixed at R, the maximum number of processes that cannot deadlock is $\lfloor (R - 1)/(m - 1) \rfloor$.
- Livelock: processes keep changing state but make no progress. Starvation: indefinite postponement without a cycle.

> **Trap:** For the Banker's algorithm, always recompute Available after each completion and check against *Need* (not Max).

7. Exam strategy & revision checklist

- Study order: memory management (paging, TLB, replacement) → process synchronization → scheduling → deadlock → file system/disk → processes/threads.
- Most OS questions are numerical: keep formulas for page-table size, EAT, disk access, inode file size, and scheduling times at your fingertips.
- ☐ Page-table size and levels. ☐ EAT with TLB and page faults. ☐ FIFO/LRU/optimal fault counts. ☐ Gantt charts for SJF/SRTF/RR. ☐ Semaphore traces. ☐ Banker's algorithm. ☐ Redo `GATE_CS_Operating_System.pdf` past questions.